SMART INDIA HACKATHON 2026 | INTERNAL HACKATHON

ROADGUARD AI

AI-Powered Smart Road Safety, Dynamic Risk Prioritization & Civic Accountability Platform

Problem Statement Code	SIH Internal Hackathon – MB-04
Theme / Category	Smart Infrastructure & Logistics
Project Name	ROADGUARD AI
Team Name	YuvaTech
Demonstration Jurisdiction	Meerut Municipal Region (UP PWD / Nagar Nigam / NHAI)
Document Release & Date	Version 1.0 (March 2026) | Full Engineering Documentation

Submitted for Smart India Hackathon Internal Evaluation & Technical Review
Team YuvaTech — Advanced Agentic & Civic Engineering Division

2. Table of Contents

3. Executive Summary

ROADGUARD AI is an advanced, production-grade civic intelligence and road asset management platform designed to solve the chronic road degradation, unsafe transit corridors, and fragmented grievance redressing mechanisms across Indian municipalities. Developed specifically for Smart India Hackathon (SIH) Internal Hackathon under Problem Statement MB-04 ('Smart Infrastructure & Logistics') by Team YuvaTech, the platform transitions municipal road maintenance from reactive, complaint-driven patching to predictive, evidence-based, and contractor-accountable lifecycle management.

Traditional civic grievance portals function as simple digital suggestion boxes. They suffer from complaint duplicates, unverified submissions, subjective severity ratings, absence of spatial clustering, and total disconnection from municipal public works contracts and Defect Liability Periods (DLP). ROADGUARD AI completely eliminates these structural gaps through an integrated six-stage technological pipeline:

- **Mobile-First PWA Reporting:** Empowers citizens to capture photographic evidence, auto-harvest GPS coordinates, select damage classifications, and submit grievances with instant client-side AI image quality feedback.
- **Resilient Offline Synchronization:** Ensures zero citizen report loss during connectivity blackouts via IndexedDB caching and background sync draining upon network re-establishment.
- **Computer Vision Road Damage Analysis:** Employs an intelligent multi-provider architecture (supporting heuristic Indian Roads Congress rules, Google Gemini Vision, and OpenAI GPT-4o) to classify pavement distress (potholes, structural cracking, waterlogging, edge erosion) and assess collision hazard.
- **Dynamic 6-Factor Risk Prioritization Engine:** Replaces raw citizen anger with an objective mathematical prioritization index (0–100) weighing Pavement Severity (25%), Direct Hazard (20%), Road Transit Importance (20%), Spatial Incident Density (15%), Historical Recurrence (10%), and SLA Urgency (10%).
- **Spatial Deduplication & Incident Clustering:** Uses Haversine spatial calculations to identify co-located complaints within an 80-meter radius, merging repetitive submissions into a single municipal work order while escalating priority density scores within a 150-meter radius.
- **Public Tender Intelligence & DLP Tracking:** Cross-references damage coordinates against active civil engineering tenders, surfacing contractor identities, defect liability warranty periods (DLP), and public expenditure details to prevent duplicate billing for under-warranty roads.
- **Before/After AI Repair Verification:** Enforces strict dual-photo proof where municipal engineers upload post-repair imagery evaluated by visual comparison algorithms before closing complaints.

> **[FACTUAL COMPLIANCE & REPOSITORY CALIBRATION]**
> *ROADGUARD AI is fully implemented in the current repository using React 18, Vite 6, Tailwind CSS, Leaflet, Node.js, Express, TypeScript, and Prisma ORM. It is calibrated with realistic geographic data from the Meerut municipal zone (Uttar Pradesh). To respect academic integrity, all demo records, simulated tenders, and heuristic analysis outputs are explicitly marked throughout this document.*

4. Problem Statement (SIH MB-04)

Road networks constitute the primary logistical backbone of economic mobility in India, facilitating over 85% of passenger transit and 70% of freight traffic. However, Indian urban roads suffer from severe structural premature degradation triggered by intense monsoon precipitation, inadequate drainage, excessive axle loading, poor compaction, and rapid asphalt binder oxidization.

According to official annual reports published by the Ministry of Road Transport and Highways (MoRTH), road accidents caused by potholes, severe road surface ruts, and unmaintained transit corridors claim thousands of innocent civilian lives annually and leave tens of thousands permanently incapacitated. Two-wheeler riders and non-motorized transport users bear the disproportionate brunt of this crisis.

In the context of Smart India Hackathon Problem Statement MB-04 ('Smart Infrastructure & Logistics'), the challenge is not merely detecting a pothole, but creating a transparent, accountable, and closed-loop civic infrastructure workflow. Municipalities currently struggle with:

- **Fragmented & Disconnected Grievance Channels:** Citizens abandon municipal portals because complaint forms are cumbersome, lack offline support, and provide zero visibility into repair progress.
- **Lack of Explainable Prioritization:** Authorities receive hundreds of disparate complaints without automated risk scoring, causing critical arterial hazards to languish while low-traffic residential issues receive arbitrary attention.
- **Duplicate Grievance Floods:** A single hazardous pothole on a major route can trigger dozens of separate complaints from different motorists, overwhelming field staff with duplicate paperwork.
- **Contractor Immunity & Missing DLP Enforcement:** Roads repaired under public contracts often deteriorate within months due to substandard execution. Because complaints are not mapped to tender contracts, public funds are repeatedly spent repairing roads that are still legally protected by contractor defect liability warranties.
- **Unverifiable 'Paper Resolutions':** Reports are routinely marked 'Resolved' on municipal portals without verifiable evidence, leading to widespread public distrust.

5. Existing System & Conventional Gaps

Civic bodies currently operate centralized citizen grievance portals such as CPGRAMS, state-level CM HelpLine systems (e.g., Jansunwai UP), municipal WhatsApp helplines, and Swachhata apps. While these systems represent commendable digital governance milestones, an objective technical analysis reveals distinct functional and structural bottlenecks:

Parameter	Conventional Municipal System	ROADGUARD AI Platform
Grievance Routing	Manual triage by municipal clerical staff based on user-typed text descriptions.	Automated geospatial snapping to road segments and responsible department (PWD, Nagar Nigam, NHAI).
Prioritization	Static First-In-First-Out (FIFO) queue or subjective VIP escalation.	Dynamic 6-Factor Risk Score (0-100) based on severity, hazard, traffic corridor, and SLA urgency.
Duplicate Handling	No automated duplicate detection; multiple officers assigned to the same pothole.	Automated Haversine spatial deduplication (<=80m) clustering identical incidents within 14 days.
Pavement Diagnostics	None; relies on vague text ('bada gaddha hai').	AI-assisted damage classification conforming to Indian Roads Congress (IRC:67 & IRC:SP:72) defect taxonomies.
Contractor Accountability	Zero link between public works tenders, contractors, and citizen complaints.	Public Tender Intelligence linking road segments to contractors and Defect Liability Periods (DLP).

Repair Verification	Self-certified text status update ('Work done') without photo verification.	Compulsory dual-photo visual improvement comparison with AI match scoring and authority audit trail.

6. Proposed Solution – ROADGUARD AI

ROADGUARD AI introduces a cohesive, closed-loop civic intelligence architecture. It unifies citizens, municipal engineers, and public procurement records into a single transparent operational ecosystem. The platform enforces an unambiguous, verifiable lifecycle from initial distress capture to final maintenance sign-off:

```
+-------------------------------------------------------------------------------------------------
---------+
|                                    ROADGUARD AI END-TO-END WORKFLOW
|
+-------------------------------------------------------------------------------------------------
---------+
  [Citizen]
      |
      |──> Live Camera / Photo Selection (Image Quality Pre-Check: Blur, Darkness, Road
Visibility)
      |──> Auto GPS Acquisition (Latitude, Longitude + Accuracy Snapping + Interactive Draggable
Pin)
      └──> Damage Description & Damage Type Selection (Pothole, Crack, Surface, Edge, Drainage)
      |
      ▼
  [AI Road Damage Diagnostic Service]
      |
      |──> Evaluates pavement failure classification & hazard severity (Low, Medium, High,
Critical)
      |──> Generates IRC-aligned remedial action recommendations (IRC:67, asphalt patch
compaction)
      └──> Quantifies Direct Road Safety Collision Risk (0 - 100)
      |
      ▼
  [Geospatial Clustering & Deduplication Engine]
      |
      |──> Haversine Proximity Check (<= 80m + matching damage type in past 14 days = DUPLICATE)
      └──> Radius Incident Density (<= 150m = Cluster Density Increment)
      |
      ▼
  [Dynamic Road Risk Prioritization Engine]
      |
      └──> Dynamic Score = 25% Severity + 20% Hazard + 20% Road Class + 15% Density + 10% Recur +
10% SLA
      |
      ▼
  [Administrative Jurisdiction & Tender Linkage]
      |
      |──> Snaps coordinates to responsible administrative department (UP PWD, Nagar Nigam, NHAI)
      └──> Maps road segment to active/awarded Public Works Tender & Defect Liability Period
(DLP)
      |
      ▼
  [Municipal Authority Operational Dashboard]
      |
      |──> Prioritized Worklist ordered by Dynamic Risk Score (CRITICAL, HIGH, MEDIUM, LOW)
```

```
      ├──> SLA Countdown Monitoring (24h Critical, 48h High, 120h Medium, 240h Low)
      └──> Work Allocation & Field Dispatch to Contractor / Maintenance Crew
      |
      ▼
  [Physical Repair & Evidence Capture]
      |
      └──> Field Engineer uploads 'After-Repair' Photographic Evidence
      |
      ▼
  [AI-Assisted Repair Verification & Audit]
      |
      ├──> Computes Location Match Confidence & Visible Surface Improvement Score
      └──> Generates Recommendation (PASS / NEEDS_REINSPECTION / INCONCLUSIVE)
      |
      ▼
  [Executive Authority Final Sign-Off & Road Health Recalculation]
      |
      ├──> Executive Engineer approves closure or orders contractor re-inspection
      └──> Road Segment Health Score (0-100) and Chronic Hotspot Registry automatically updated
+-------------------------------------------------------------------------------------------
---------+
```

7. Core Engineering & Civic Objectives

The ROADGUARD AI platform was engineered to satisfy eight measurable civic and technological objectives:

- **1. Streamlined Citizen Participation:** Provide a zero-friction, mobile-optimized progressive web application enabling citizens to report road hazards in under 30 seconds with automatic GPS geocoding and live optical quality feedback.
- **2. Objective Hazard Prioritization:** Replace subjective human triage with an explainable multi-factor algorithmic risk score that directs municipal resources to the most dangerous hazards first.
- **3. Automated Duplicate Mitigation:** Eliminate redundant municipal work orders through automated spatial clustering, aggregating multi-citizen complaints into unified incident tickets.
- **4. Public Procurement Accountability:** Cross-reference municipal grievances with public civil tenders, identifying roads under Defect Liability Periods to enforce contractor warranty rectifications without spending public funds.
- **5. Enforced SLA Governance:** Establish transparent, severity-calibrated Service Level Agreement (SLA) timers with visual overdue warnings to hold executive officers accountable.
- **6. Evidence-Based Verification:** Prevent fake complaint closures through dual-photo before/after visual verification analyzed by computer vision and signed off by authorized engineers.
- **7. Preventive Road Asset Management:** Continuously compute a dynamic health index (0–100) for every arterial road, enabling city administrations to transition from reactive patching to scheduled preventive resurfacing.
- **8. Inclusive Offline-First Architecture:** Ensure high platform usability in rural and low-bandwidth urban transit pockets via full offline IndexedDB draft storage and background sync.

8. Key Features & Implementation Matrix

The following matrix details all functional modules implemented in ROADGUARD AI, contrasting operational features with planned future enhancements:

Feature Module	Functional Capability & Technical Mechanism	Status / Module

Citizen Road Reporting	Mobile-optimized reporting interface with damage classification, description, and client-side validation.	Implemented / Frontend
Camera & Gallery Input	Direct device camera capture or local photo selection with real-time thumbnail preview and zoom modal.	Implemented / Frontend
Geospatial Geocoding	HTML5 Geolocation API with accuracy meter, reverse address resolution, and interactive draggable Leaflet pin.	Implemented / Frontend & Leaflet
Interactive City Map	Dynamic OpenStreetMap with color-coded severity markers, cluster tooltips, and road segment overlays.	Implemented / Frontend & Leaflet
AI Image Quality Check	Pre-submission optical inspection evaluating blur, darkness, road presence, and technical clarity score.	Implemented (Demo Heuristic + Cloud API)
AI Damage Classification	IRC-aligned classification (pothole, alligator crack, surface erosion, waterlogging, edge scour).	Implemented (Demo Heuristic + Cloud API)
Dynamic Road Risk Engine	Multi-factor algorithm (0-100) scoring severity, safety risk, transit corridor class, density, recurrence, and SLA.	Implemented / Backend Priority
Spatial Deduplication	Haversine proximity check (<=80m + matching damage type in 14 days) merging duplicate reports.	Implemented / Backend Duplicate
Incident Clustering	Spatial radius clustering (<=150m) aggregating report density to detect localized structural corridors.	Implemented / Backend Duplicate
Road Segment Health	Segment-level health index (0-100) factoring open complaints, critical failures, and historical recurrence.	Implemented / Backend Health
Authority Dashboard	Role-gated operational console with KPI metrics, risk-sorted worklists, and workflow progression tools.	Implemented / Frontend Authority
SLA Urgency Tracking	Dynamic resolution countdowns (24h to 240h) based on severity, with automated overdue alerts.	Implemented / Backend SLA
Before/After Photo Verification	Side-by-side photographic evidence viewer enforcing before/after visual proof for all closed repairs.	Implemented / Frontend & Backend
AI Repair Verification	Visual improvement scoring (0-100), location match confidence, and automated PASS/REINSPECT recommendation.	Implemented / Backend Verification
Public Tender Intelligence	Cross-references road segments against public procurement tenders, contractors, and Defect Liability Periods.	Implemented (Demo/Synthetic Dataset)
JWT Authentication & RBAC	Role-Based Access Control enforcing CITIZEN, AUTHORITY, and ADMIN role boundaries using bcrypt & JWT.	Implemented / Backend Auth

PWA & Offline Capability	Service worker precaching (39 assets) + IndexedDB offline draft storage with auto-sync on reconnect.	Implemented / PWA & Service Worker
Cloud & Local Storage	Pluggable file storage supporting Cloudinary, Supabase Storage buckets, and local disk fallback.	Implemented / Backend Storage
On-Device Edge Vision (YOLOv8)	Real-time edge neural network inference directly on browser canvas via WebAssembly/TensorFlow.js.	Future Enhancement / Planned Feature
Automated Municipal ERP Sync	Direct webhook integration with state municipal ERPs (e.g., e-NagarSewa UP, CPGRAMS API).	Future Enhancement / Planned Feature
Accelerometer Pothole Sensing	Continuous vehicular telematics detecting road anomalies via smartphone gyroscope & accelerometer.	Future Enhancement / Planned Feature

9. User Roles & Access Control

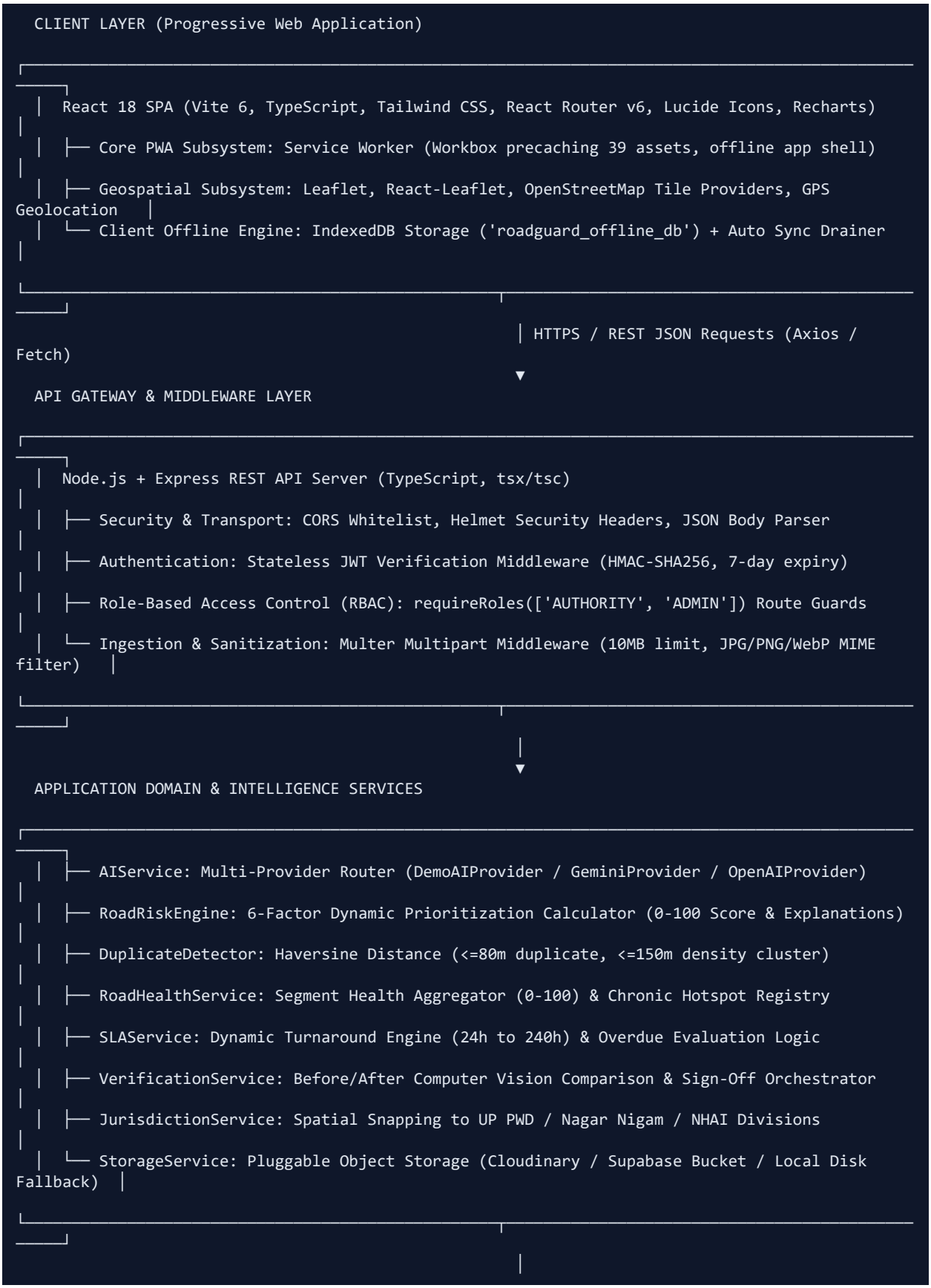
ROADGUARD AI enforces strict Role-Based Access Control (RBAC) defined in the Prisma schema and enforced via Express middleware (`requireRoles`). Three distinct actor roles exist within the system:

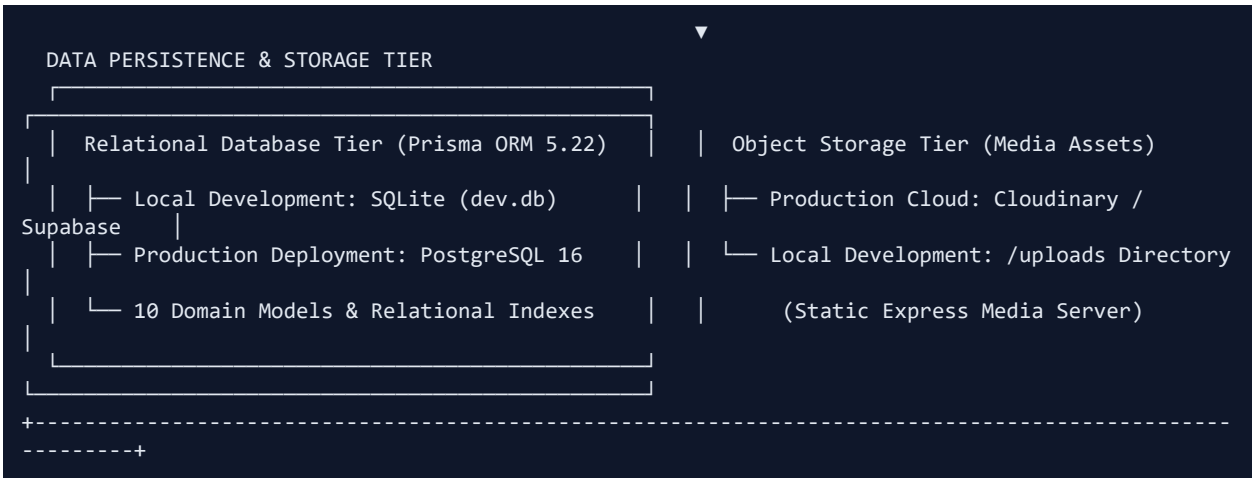
- **1. Citizen (Role: CITIZEN):** Any member of the public can register with an email and phone number. Capabilities: Submit geo-tagged road damage complaints, upload photos, receive AI quality and damage feedback, view personal complaint history (`/my-reports`), track real-time resolution timelines, view public city road health, and inspect public tender data. Permissions: Read-only access to city reports, write access only to self-authored drafts and complaints.
- **2. Municipal Authority / Engineer (Role: AUTHORITY):** Designated municipal officers (e.g., Executive Engineers, Assistant Engineers, Junior Engineers) attached to specific departments (UP PWD, Nagar Nigam, NHAI). Capabilities: Access the Authority Operations Dashboard (`/authority`), inspect the priority-sorted complaint queue, filter by jurisdiction and severity, transition report status (IN_REVIEW -> ASSIGNED -> IN_PROGRESS -> REPAIRED), upload official 'After-Repair' photographic proof, trigger AI repair verification, and provide human-in-the-loop closure approvals.
- **3. System Administrator (Role: ADMIN):** Departmental system administrators overseeing municipal zones. Capabilities: Manage department registries, configure default SLA turnaround targets, manage road segment spatial definitions, review audit logs, and oversee tender intelligence linkages.

10. End-to-End System Architecture

ROADGUARD AI adopts a decoupled, multi-tier client-server architecture engineered for horizontal scalability, cloud-agnostic deployment, and high operational resilience. The architecture consists of four distinct operational layers:

```
+-------------------------------------------------------------------------------------------------
---------+
|                                    ROADGUARD AI SYSTEM ARCHITECTURE
|
+-------------------------------------------------------------------------------------------------
---------+
```


CLIENT LAYER (Progressive Web Application)
React 18 SPA (Vite 6, TypeScript, Tailwind CSS, React Router v6, Lucide Icons, Recharts)
Core PWA Subsystem: Service Worker (Workbox precaching 39 assets, offline app shell)
Geospatial Subsystem: Leaflet, React-Leaflet, OpenStreetMap Tile Providers, GPS Geolocation
Client Offline Engine: IndexedDB Storage ('roadguard_offline_db') + Auto Sync Drainer
HTTPS / REST JSON Requests (Axios / Fetch)
API GATEWAY & MIDDLEWARE LAYER
Node.js + Express REST API Server (TypeScript, tsx/tsc)
Security & Transport: CORS Whitelist, Helmet Security Headers, JSON Body Parser
Authentication: Stateless JWT Verification Middleware (HMAC-SHA256, 7-day expiry)
Role-Based Access Control (RBAC): requireRoles(['AUTHORITY', 'ADMIN']) Route Guards
Ingestion & Sanitization: Multer Multipart Middleware (10MB limit, JPG/PNG/WebP MIME filter)
APPLICATION DOMAIN & INTELLIGENCE SERVICES
AIService: Multi-Provider Router (DemoAIProvider / GeminiProvider / OpenAIProvider)
RoadRiskEngine: 6-Factor Dynamic Prioritization Calculator (0-100 Score & Explanations)
DuplicateDetector: Haversine Distance (<=80m duplicate, <=150m density cluster)
RoadHealthService: Segment Health Aggregator (0-100) & Chronic Hotspot Registry
SLAService: Dynamic Turnaround Engine (24h to 240h) & Overdue Evaluation Logic
VerificationService: Before/After Computer Vision Comparison & Sign-Off Orchestrator
JurisdictionService: Spatial Snapping to UP PWD / Nagar Nigam / NHAI Divisions
StorageService: Pluggable Object Storage (Cloudinary / Supabase Bucket / Local Disk Fallback)



11. Comprehensive Technology Stack

Every technology utilized in ROADGUARD AI was chosen based on production stability, developer velocity, type-safety, and offline resilience:

Layer / Component	Technology / Library	Version	Technical Purpose in ROADGUARD AI
Frontend Framework	React	18.3.1	Declarative component-based UI rendering with Virtual DOM.
Frontend Build Tool	Vite	6.0.3	Ultra-fast ESM bundler with Hot Module Replacement and production rollup.
Language (Full Stack)	TypeScript	5.7.2	Static type checking across frontend, backend, and shared domain models.
Styling Framework	Tailwind CSS	3.4.16	Utility-first CSS framework providing sleek modern dark/light styling.
Icons & Visuals	Lucide React	0.468.0	Tree-shakable SVG civic iconography for road hazards and dashboards.
Interactive Maps	Leaflet & React-Leaflet	1.9.4 / 4.2.1	Mobile-optimized slippy mapping with OpenStreetMap tiles.
Charts & Analytics	Recharts	2.15.0	Declarative SVG charting library for road health distribution and trends.
PWA Subsystem	vite-plugin-pwa & Workbox	1.3.0	Service Worker generation, manifest management, and 39-asset precaching.
Client Offline DB	IndexedDB API	Native W3C	Browser-native transactional NoSQL storage for offline report drafting.

Backend Runtime	Node.js	20.x LTS	High-throughput asynchronous event-driven JavaScript server runtime.
HTTP Server Framework	Express	4.21.2	Lightweight, unopinionated routing framework for RESTful API endpoints.
Database ORM	Prisma ORM	5.22.0	Type-safe database client, schema migrations, and parameterized query execution.
Relational Databases	SQLite / PostgreSQL	3.x / 16.x	SQLite for local zero-config dev; PostgreSQL for production cloud deployment.
File Uploads	Multer	1.4.5-lts.1	Streaming multipart/form-data handler with 10MB memory and disk limits.
Authentication & Security	bcryptjs & jsonwebtoken	2.4.3 / 9.0.2	Secure password hashing (10 salt rounds) and stateless signed JWTs.
Data Validation	Zod	3.24.1	Runtime schema validation for AI JSON inference and client request payloads.
Cloud Object Storage	Cloudinary & Supabase SDKs	REST APIs	Off-server persistent media hosting for citizen before/after photographs.
Testing Suite	Jest & ts-jest	29.7.0 / 29.2.5	Unit and integration testing framework for API routes and priority engines.
Cloud Deployment	Render Blueprint (render.yaml)	Current Spec	Infrastructure-as-Code multi-service deployment for Node, PWA, and Postgres.

12. Frontend Architecture & Modular Design

The ROADGUARD AI frontend is organized as a single-page application (SPA) structured around modular functional domains. The application emphasizes responsive mobile-first ergonomics, reactive state synchronization, and progressive offline enhancement:

- **Modular Component Hierarchy:** The application lifecycle is anchored by React 18 Concurrent features and structured routing via `react-router-dom` v6. Routes are partitioned into public discovery views (`/`, `/map`, `/tenders`, `/road-health`), citizen workflows (`/report`, `/my-reports`, `/reports/:id`), and restricted administrative consoles (`/authority`).
- **State & Context Architecture:** Global session state is governed via `AuthContext.tsx`, exposing reactive user profiles, login/logout actions, and role flags (`isAuthority`, `isAdmin`). Real-time network connectivity is observed via window event listeners (`online`/`offline`), propagating state to the `offlineSync` service.
- **API Abstraction Layer:** Located in `src/services/api.ts`, a centralized HTTP client wraps browser Fetch calls, automatically attaching Bearer JWT tokens, intercepting authentication failures, and standardizing server error responses.

- **Geospatial Mapping Subsystem:** Built using Leaflet and React-Leaflet with custom SVG marker icons. Supports hardware GPS acquisition, interactive draggable marker positioning, and zoom boundaries focused on Meerut coordinates (`[28.9845, 77.7064]`).
- **Offline Storage & Background Sync:** Engineered in `src/services/offlineSync.ts` using the browser IndexedDB API (`roadguard_offline_db`). When offline, report submissions are stored locally with base64 image data and queued with `PENDING_SYNC` status. Upon reconnect, an automatic background drainer uploads photos, submits reports, and broadcasts live progress.

Actual Frontend Directory Structure:

```
frontend/
├── public/
│   ├── favicon.ico
│   ├── icon-192.png
│   ├── icon-512.png
│   └── manifest.webmanifest      # PWA App Manifest
├── src/
│   ├── assets/                   # Static logos and graphic assets
│   ├── components/
│   │   ├── ai/                   # AI Image Quality Preview & Diagnostic Cards
│   │   ├── dashboard/            # Authority Metric KPI Cards & Priority Tables
│   │   ├── layout/               # Navbar, Footer, Mobile Navigation Bar
│   │   ├── map/                  # Leaflet MapContainer, DraggablePin, ClusterMarkers
│   │   ├── offline/              # Offline Indicator Banner & Sync Status Badge
│   │   ├── pwa/                  # PWA Install Prompt & Service Worker Update Banner
│   │   ├── reports/              # Report Cards, Filter Bars, Damage Selectors
│   │   ├── tender/               # Public Tender Cards & DLP Warranty Badges
│   │   ├── timeline/             # Status Timeline Event Flow & Activity Trackers
│   │   ├── ui/                   # Modal, Button, Badge, Card, Spinner Atoms
│   │   ├── upload/               # Camera/Gallery Selector, Image Zoom Modal
│   │   └── verification/         # Dual-Photo Before/After Slider & Match Scorecard
│   ├── context/
│   │   └── AuthContext.tsx       # Global JWT Session & Role State
│   ├── pages/
│   │   ├── LandingPage.tsx       # Public Civic Hero & Feature Showcase
│   │   ├── LoginPage.tsx         # Citizen & Authority Login with Demo Quick-Buttons
│   │   ├── RegisterPage.tsx      # User Registration Form
│   │   ├── CreateReportPage.tsx  # Mobile-First Damage Reporting Workflow
│   │   ├── MyReportsPage.tsx     # Citizen Personal Dashboard & Offline Sync Queue
│   │   ├── ReportDetailPage.tsx  # In-Depth Complaint Inspection & Evidence Viewer
│   │   ├── AuthorityDashboard.tsx# Municipal Engineer Operations Console
│   │   ├── PublicMapPage.tsx     # Full-Screen Interactive City Incident Map
│   │   ├── TenderIntelligencePage.tsx # Public Civil Tenders & DLP Matrix
│   │   └── RoadHealthPage.tsx    # Municipal Road Segment Health Scorecards
│   ├── services/
│   │   ├── api.ts                # Centralized REST Client with JWT Interceptors
│   │   └── offlineSync.ts        # IndexedDB Storage Engine & Auto Sync Drainer
│   ├── App.tsx                   # Route Definitions & Provider Wiring
│   ├── index.css                 # Tailwind Directives & Custom Scrollbars
│   └── main.tsx                  # React DOM Root Entrypoint
├── vite.config.ts                # Vite Configuration with VitePWA Plugin
└── package.json                  # Frontend Dependencies & Scripts
```

13. Backend Architecture & Controller Pipeline

The ROADGUARD AI backend is an enterprise-grade RESTful API service built on Node.js, Express, and TypeScript. The application enforces a strict separation of concerns across Routes, Controllers, Domain Services, and Data Access Layers:

- **Express Server Core:** Centralized in `src/server.ts`. Configures CORS, sets up JSON body parsers, mounts static media handlers for `/uploads`, binds API routers under `/api/*`, and attaches a centralized error handling middleware that intercepts Multer file size errors, Zod validation issues, and database constraint violations.
- **Modular Route Dispatchers:** Organized in `src/routes/` with clean domain partitioning: `authRoutes`, `reportRoutes`, `uploadRoutes`, `roadHealthRoutes`, `tenderRoutes`, `verificationRoutes`, `departmentRoutes`, `analyticsRoutes`, and `notificationRoutes`.
- **Controller Layer:** Implemented in `src/controllers/`. Responsible for extracting query/body parameters, validating payloads with Zod schemas, orchestrating business domain calls, and returning consistent HTTP JSON envelopes (`{ success: true, data: ... }`).
- **Domain Service Layer:** Encapsulates all civic engineering business logic into isolated, testable single-responsibility classes (`AIService`, `RoadRiskEngine`, `DuplicateDetector`, `RoadHealthService`, `SLAService`, `VerificationService`, `JurisdictionService`, `StorageService`).
- **Database & ORM Tier:** Powered by Prisma Client (`src/config/prisma.ts`), providing type-safe parameterized SQL execution, automated connection pooling, and resilient transaction management.

Actual Backend Directory Structure:

```
backend/
├── prisma/
│   ├── schema.prisma             # Complete Relational Database Schema (10 Models)
│   └── seed.ts                   # Authentic Meerut Calibration & Synthetic Seed Data
├── scripts/
│   └── prepare-prisma.js         # Cross-Platform SQLite/PostgreSQL Switcher Script
├── src/
│   ├── config/
│   │   └── prisma.ts             # Prisma Client Singleton Instance
│   ├── controllers/
│   │   ├── analyticsController.ts# Municipal Analytics KPI Aggregators
│   │   ├── authController.ts     # User Registration, Login & Profile Handlers
│   │   ├── reportController.ts   # Report Ingestion, Triage & Status Progression
│   │   ├── roadHealthController.ts # Road Segment Health Metrics & Hotspot Queries
│   │   ├── tenderController.ts   # Public Civil Tender Directory Queries
│   │   └── verificationController.ts # AI Repair Verification & Decision Handlers
│   ├── middleware/
│   │   ├── authMiddleware.ts     # JWT Extraction & Role Guarding (requireRoles)
│   │   └── errorMiddleware.ts    # Centralized HTTP Error & Exception Interceptors
│   ├── routes/
│   │   ├── analyticsRoutes.ts    # /api/analytics
│   │   ├── authRoutes.ts         # /api/auth
│   │   ├── departmentRoutes.ts   # /api/departments
│   │   ├── notificationRoutes.ts # /api/notifications
│   │   ├── reportRoutes.ts       # /api/reports
│   │   ├── roadHealthRoutes.ts   # /api/road-health
│   │   ├── tenderRoutes.ts       # /api/tenders
│   │   ├── uploadRoutes.ts       # /api/upload
│   │   └── verificationRoutes.ts # /api/verification
│   ├── services/
│   │   ├── ai/                   # AIService, DemoAIProvider, Gemini, OpenAI, Schemas
│   │   ├── duplicate/            # DuplicateDetector (Haversine & Radius Clustering)
│   │   ├── jurisdiction/         # JurisdictionService (Spatial Department Snapping)
```

```
│   │   ├── notification/        # NotificationService (In-App Alert Dispatcher)
│   │   ├── priority/            # RoadRiskEngine (6-Factor Mathematical Prioritizer)
│   │   ├── roadHealth/          # RoadHealthService (Health Index Recalculator)
│   │   ├── sla/                 # SLAService (Target Hours & Overdue Computations)
│   │   ├── storage/             # StorageService (Cloudinary / Supabase / Local)
│   │   ├── tender/              # TenderService (Contractor & DLP Matrix)
│   │   └── verification/        # VerificationService (Before/After AI Inspection)
│   └── server.ts                # Express Server Bootstrapper & Static Host
├── tests/
│   ├── ai.test.ts               # AI Provider Schema & IRC Output Tests
│   ├── api.test.ts              # Core HTTP API Endpoint Tests
│   └── priority.test.ts         # Road Risk Engine Weighting & Ranking Tests
└── package.json                 # Backend Scripts, Dependencies & Jest Config
```

14. Relational Database Design & Schema Models

The database schema is defined in Prisma ORM (`backend/prisma/schema.prisma`). It comprises 10 relational entities supporting multi-role authentication, municipal hierarchy, road segments, public tenders, citizen reports, AI diagnostics, dynamic priority breakdowns, dual-photo repair verification, and timeline audits:

Entity Name	Primary Purpose & Role in System	Key Fields & Relationships
User	Stores registered citizens, municipal engineers, and admins.	id, email, passwordHash, name, role (CITIZEN/AUTHORITY/ADMIN), departmentId, reports, assignedReports.
Department	Municipal bodies (UP PWD, Nagar Nigam Meerut, NHAI PIU).	id, code (e.g. PWD_MRT), name, jurisdiction, nodalOfficer, contactEmail, slaDaysDefault, roadSegments, tenders.
RoadSegment	Physical road corridors (e.g. Jail Chungi Road, MRT-RD-01).	id, code, name, startLat/Lng, endLat/Lng, lengthKm, departmentId, healthScore (0-100), recurringDamageCount, isRecurringHotspot.
Tender	Public works civil procurement contracts & DLP warranties.	id, tenderId, roadName, workDescription, departmentId, roadSegmentId, tenderValue, workPeriod, tenderStatus, contractor, contractorStatus, DLP.
RoadReport	Core incident entity storing citizen damage complaints.	id, clientReportId, userId, imageUrl, additionalImages, evidenceSource, lat, lng, address, damageType, severity, status, riskScore, isDuplicate, slaDueAt.
AIAnalysis	Computer vision diagnostic results & IRC action steps.	id, reportId, damageType, severity, confidence, roadSafetyRisk, description, recommendedAction, isAcceptableQuality, qualityScore, isBlurry, isTooDark.
PriorityAssessment	Mathematical breakdown of 6-factor	id, reportId, overallScore (0-100), riskLevel

	risk score.	(LOW/MED/HIGH/CRIT), severityScore, safetyRiskScore, densityScore, roadImportanceScore, recurrenceScore, slaUrgencyScore, explanation.
RepairVerification	Dual-photo before/after repair inspection records.	id, reportId, beforeImageUrl, afterImageUrl, locationMatchConfidence, visibleImprovementScore, remainingDamageScore, overallConfidence, recommendation, authorityDecision.
StatusTimelineEvent	Immutable audit trail of complaint lifecycle changes.	id, reportId, status, label, description, timestamp, actorRole, actorName, notes.
Notification	In-app notifications for citizens and municipal staff.	id, userId, title, message, reportId, isRead, type (INFO, ASSIGNMENT, SLA_ALERT, VERIFICATION, RESOLUTION).
SyncQueueMetadata	Tracks client-side offline sync attempts and retries.	id, clientReportId, status (PENDING_SYNC/SYNCING/SYNCED/FAILED), attempts, lastError.

15. REST API Documentation & Endpoints

The backend exposes a structured, RESTful API interface operating over JSON payloads and multipart media uploads. All protected routes require a standard Bearer JWT token passed in the `Authorization` HTTP header:

Method	Endpoint Route	Purpose & Operation	Auth / Roles	Request / Response Summary
POST	/api/auth/register	Register new citizen or engineer.	Public	Body: { email, password, name, phone, role } -> Token & User profile
POST	/api/auth/login	Authenticate existing user session.	Public	Body: { email, password } -> Token, User profile, role flags
GET	/api/auth/profile	Retrieve active authenticated profile.	JWT Required	Headers: Bearer <token> -> User entity with department details
GET	/api/reports	List all public road damage reports.	Public	Query: status, severity, departmentId, roadSegmentId -> Array of RoadReports

GET	/api/reports/my-reports	List citizen's personal report history.	JWT (Citizen)	Returns user's submitted reports with status, timeline, and risk breakdown
GET	/api/reports/:id	Retrieve complete report dossier.	Public	Params: reportId -> Full report with AIAnalysis, Priority, Verification, Timeline
POST	/api/reports	Submit new road hazard complaint.	JWT (Citizen)	Body: { imageUrl, lat, lng, address, damageTypeHint, description } -> Created Report
PATCH	/api/reports/:id/status	Advance report maintenance status.	JWT (Authority)	Body: { status, notes } -> Updates status, generates timeline audit event
POST	/api/reports/:id/after-photo	Upload contractor after-repair photo.	JWT (Authority)	Body: { afterImageUrl } -> Saves repairAfterImageUrl, triggers verification
POST	/api/upload	Upload photographic evidence file.	Public / Citizen	Multipart: photo/image (<=10MB) -> Returns { url, filename, storageProvider }
GET	/api/road-health	Fetch all road segment health scores.	Public	Returns road segments sorted by health score with open/critical counts
GET	/api/road-health/hotspots	Fetch recurring damage hotspot list.	Public	Returns road segments flagged with isRecurringHotspot = true
GET	/api/road-health/resolve	Resolve jurisdiction from coordinates.	Public	Query: lat, lng -> Nearest road segment, department, and active tender
GET	/api/road-health/:id	Retrieve detailed road segment dossier.	Public	Params: segmentId -> Road segment with historical complaint distribution
GET	/api/tenders	List all public works tender records.	Public	Returns procurement tenders with DLP

				warranty and contractor status
GET	/api/tenders/:id	Fetch specific tender contract details.	Public	Params: tenderId -> Tender record linked to road segment and department
POST	/api/verification/run	Execute AI repair comparison on demand.	JWT (Authority)	Body: { reportId, afterImageUrl } -> AI before/after match scorecard
POST	/api/verification/:id/decision	Submit final human authority verdict.	JWT (Authority)	Body: { decision: 'APPROVED' | 'REJECTED_REINSPECT', notes } -> Final status
GET	/api/departments	List municipal department authorities.	Public	Returns UP PWD, Nagar Nigam, NHAI with active workload counts
GET	/api/analytics/summary	Retrieve high-level municipal KPIs.	Public	Returns totalReports, resolvedReports, criticalCount, avgRiskScore
GET	/api/notifications	Retrieve citizen/officer notifications.	JWT Required	Returns in-app alert notifications for logged-in user
PATCH	/api/notifications/:id/read	Mark specific notification as read.	JWT Required	Params: notificationId -> Updates isRead = true

16. Authentication, Role Security & Data Protection

Security in ROADGUARD AI is implemented through defense-in-depth across the client, network, application, and database tiers:

- **Cryptographic Password Hashing:** User passwords are encrypted prior to database persistence using bcryptjs with 10 salt rounds. Cleartext credentials are never written to logs or transmitted across API responses.
- **Stateless JWT Token Issuance:** Sessions are managed via compact, stateless JSON Web Tokens (JWT) signed with HMAC-SHA256 (`HS256`). Tokens carry user identity (`id`), email, role (`CITIZEN`, `AUTHORITY`, `ADMIN`), and department affiliation, expiring automatically after 7 days.
- **Role-Based Route Interception:** Sensitive administrative actions (such as status progression, after-repair evidence uploads, and verification verdicts) are guarded by `requireRoles(['AUTHORITY', 'ADMIN'])` middleware. Unauthorized access attempts yield immediate HTTP 403 Forbidden envelopes.
- **SQL Injection Elimination:** All database queries utilize Prisma Client parameterized statements, fully insulating the data layer from SQL injection vulnerabilities.
- **File Upload Sanitization:** The `/api/upload` endpoint implements strict Multer disk-storage rules, restricting file sizes to 10MB and enforcing strict MIME filters (`image/jpeg`, `image/png`, `image/webp`, `image/svg+xml`). Arbitrary file extensions or executable binaries are rejected with explicit HTTP 400 errors.
- **Zero Secret Exposure Policy:** Zero private keys, production database credentials, or third-party API secrets are hardcoded in client source code. Production configuration is injected exclusively via environment variables (`JWT_SECRET`, `DATABASE_URL`, `CLOUDINARY_API_KEY`, etc.).

17. AI / Intelligence Module & Multi-Provider Architecture

The AI module in ROADGUARD AI is engineered around a flexible Strategy Pattern interface (`AIProvider`), allowing the system to operate flawlessly in disconnected hackathon demo environments, on edge nodes, or connected to state-of-the-art multimodal large vision models:

- **1. Standardized AIProvider Interface:** Defines a clean contract requiring two core capabilities: `analyzeRoadDamage(input)` and `verifyRepair(input)`. All provider outputs are validated at runtime against strict Zod schemas (`AIAnalysisOutputSchema` and `AIVerificationOutputSchema`).
- **2. DemoAIProvider (Offline/Hackathon Mode):** The primary out-of-the-box provider. Runs offline without external network or API key dependencies. Analyzes image metadata and linguistic descriptions using deterministic heuristic rules aligned with Indian Roads Congress (IRC:67 for signage and IRC:SP:72 for rural/urban asphalt design). Evaluates simulated image blur, darkness, road presence, damage type, and assigns technical civil repair recommendations.
- **3. GeminiProvider (Cloud Multimodal API):** Integrates Google's `gemini-1.5-flash` or `gemini-1.5-pro` multimodal vision API. Formulates structured multimodal prompts with zero-shot civil engineering guidance, instructing the model to return strictly typed JSON evaluating pavement distress and collision risk.
- **4. OpenAIProvider (Cloud Multimodal API):** Integrates OpenAI's `gpt-4o` vision model via OpenAI SDK JSON schema structured outputs, enforcing exact compliance with the platform's diagnostic schema.
- **5. Optical Quality Assurance Engine:** Client-side and server-side algorithms evaluate whether the uploaded photograph exhibits acceptable quality: detecting extreme darkness, severe blur, or missing road surfaces, warning citizens before they submit invalid imagery.

> **[AI ARCHITECTURE & HONEST TECHNICAL DISCLOSURE]**
> *CRITICAL AI DISCLOSURE: In offline local development and hackathon demonstration modes, ROADGUARD AI operates using DemoAIProvider. It uses rigorous IRC civil engineering heuristics rather than an on-device*

> *convolutional neural network (CNN). Future integration with on-device YOLOv8/ONNX models is planned.*

18. Dynamic Road Risk Prioritization Engine

Conventional municipal systems treat all potholes equally, resulting in arbitrary repair schedules. ROADGUARD AI's `RoadRiskEngine` implements a multi-parameter mathematical formula (scaled 0 to 100) that calculates an objective dynamic risk score for every incident:

Dynamic Road Risk Mathematical Formulation:

```
Overall Risk Score (0 - 100) =
    (Severity Score        × 0.25) +
    (Safety Risk Score     × 0.20) +
    (Road Importance Score × 0.20) +
    (Cluster Density Score × 0.15) +
    (Recurrence Score      × 0.10) +
    (SLA Urgency Score     × 0.10)

Clamped: Math.min(100, Math.max(1, Math.round(RawTotal)))
```

Weighting Parameter Breakdown & Mathematical Scaling:

- **1. Pavement Severity (Weight: 25%):** Scaled to 25 points. Pavement failure depth and width. Critical = 100% (25 pts), High = 75% (18.8 pts), Medium = 50% (12.5 pts), Low = 25% (6.3 pts).
- **2. Road Safety Risk (Weight: 20%):** Scaled to 20 points. Direct collision, loss-of-control, or vehicle chassis damage hazard index (0-100) derived from AI analysis scaled by AI confidence.
- **3. Road Importance (Weight: 20%):** Scaled to 20 points. Classification of the transit corridor. HIGHWAY = 100% (20 pts), ARTERIAL = 85% (17 pts), MAJOR_DISTRICT = 65% (13 pts), LOCAL = 40% (8 pts). Critical arteries with heavy passenger loads receive higher priority.
- **4. Cluster Density (Weight: 15%):** Scaled to 15 points. Proximity density. Number of active complaints within a 150-meter radius (`nearbyReportsCount × 25`, clamped between 10 and 100). Highlights concentrated structural failure corridors.
- **5. Recurrence Factor (Weight: 10%):** Scaled to 10 points. Historical chronic failure index. If the road segment is an identified recurring hotspot, base score = 100% (10 pts); if road health is below 50, minimum 80% (8 pts); else baseline 20% (2 pts).
- **6. SLA Urgency (Weight: 10%):** Scaled to 10 points. Ratio of elapsed hours since reporting against target SLA hours (`hoursSinceReported / slaTargetHours × 100`). Escalates automatically as deadlines approach.

Prioritization Tiers & Explainable Diagnostic Output:

- CRITICAL (Score >= 80) -> Immediate emergency dispatch required within 24 hours.
- HIGH (Score >= 65 to 79) -> Arterial or high-severity defect requiring work order within 48 hours.
- MEDIUM (Score >= 45 to 64) -> Secondary collector or moderate pavement defect; SLA target 120 hours (5 days).
- LOW (Score < 45) -> Minor surface cosmetic defect on local street; SLA target 240 hours (10 days).
- Explainability Engine: Rather than outputting a black-box number, the engine outputs an array of clear civil engineering rationale strings displayed directly on the Authority Dashboard (e.g., 'Critical arterial transit corridor with high passenger density', 'Cluster detected: 3 reports within 150m radius').

19. Duplicate Detection & Incident Spatial Clustering

Duplicate complaints represent a major administrative drain on civic bodies. ROADGUARD AI addresses this through mathematical spatial deduplication in `src/services/duplicate/duplicateDetector.ts` using the Haversine great-circle distance formula:

```
Haversine Formula:
  a = sin²(Δϕ/2) + cos(ϕ1) · cos(ϕ2) · sin²(Δλ/2)
  c = 2 · atan2(√a, √(1−a))
  Distance (meters) = R · c   (where R = 6,371,000 meters)
```

Deduplication & Clustering Rules:

- **1. Temporal Window:** The detector queries active, unresolved complaints registered within the preceding 14 calendar days.
- **2. Proximity Duplicate Threshold (<= 80m):** If a new submission is located within 80 meters of an active complaint AND shares the same damage classification (or 'other'), it is flagged with `isDuplicate = true` and linked via `duplicateOfId` to the primary report. Field engineers work on a single consolidated ticket while all citizens receive tracking updates.
- **3. Spatial Density Clustering (<= 150m):** All unresolved reports within a 150-meter radius increment the cluster density counter (`nearbyReportsCount`). This elevates the report's Dynamic Road Risk Score, alerting authorities that an entire corridor is deteriorating.

20. Road Segment Health & Lifecycle Deterioration Tracking

Rather than managing isolated pothole complaints, ROADGUARD AI aggregates data at the road corridor level via `src/services/roadHealth/roadHealthService.ts`. Every physical road segment is assigned a dynamic Health Index (0 to 100):

```
Road Segment Health Score Formula:
  HealthScore = 100 - (OpenComplaints × 4) - (CriticalComplaints × 8) - (IsRecurringHotspot ? 15
: 0)
  Clamped: Math.max(15, Math.min(100, HealthScore))
```

Corridor Health Metrics & Lifecycle Intelligence:

- **Recurring Hotspot Detection:** If a road segment records 3 or more complaints within a 6-month window, the system automatically flags `isRecurringHotspot = true`. This indicates deep structural subgrade failure rather than superficial surface wear.
- **Surveillance Coverage Estimation:** Evaluates citizen reporting density per kilometer (`totalComplaints / lengthKm`). Categorized as LIMITED (< 0.8 / km), MODERATE (0.8 - 3.0 / km), or HIGH (> 3.0 / km) to highlight surveillance blindspots.
- **Preventive Resurfacing Triggers:** When a road segment's health score drops below 50, municipal leadership is prompted to schedule full bituminous overlay resurfacing rather than spending funds on repetitive cold-mix emergency patches.

21. AI-Assisted Repair Verification & Human-in-the-Loop Audit

To eradicate fraudulent 'paper closures', ROADGUARD AI enforces an evidence-based dual-photo verification workflow orchestrated by `src/services/verification/verificationService.ts`:

- **1. Mandatory After-Repair Upload:** When field maintenance is completed, the supervising engineer must upload a genuine photograph of the repaired pavement section (`/api/reports/:id/after-photo`).
- **2. Computer Vision Inspection:** The verification engine evaluates the Before and After imagery across three quantitative metrics: Location Match Confidence (0-100), Visible Surface Improvement Score (0-100), and Remaining Residual Damage Score (0-100).
- **3. Algorithmic Recommendation:** Based on inspection thresholds, the system generates an algorithmic recommendation: PASS (acceptable patch and compaction), NEEDS_REINSPECTION (visible depressions or cracks remain), or INCONCLUSIVE (poor lighting or different camera angle).
- **4. Human-in-the-Loop Authority Verdict:** The AI recommendation does NOT unilaterally close the complaint. The Executive Engineer must review the side-by-side evidence on the dashboard and record an explicit decision: APPROVED or REJECTED_REINSPECT.

> **[EVIDENCE INTEGRITY & VERIFICATION DISCLAIMER]**
> *EVIDENCE INTEGRITY POLICY: In the demonstration environment, before-repair photos utilize authentic, reusable CC-licensed road photographs from Wikimedia Commons, while simulated after-repair imagery is clearly labeled as DEMO AI ANALYSIS. In production, field photographs are captured directly via device camera.*

22. Public Tender Intelligence & Contractor Accountability

The Tender Intelligence module bridges citizen complaints with public procurement accountability. In traditional governance, contractors who execute poor-quality road work are rarely held liable because municipal engineers lack immediate access to contract warranty terms.

- **Tender-to-Asset Mapping:** Road segments are mapped to public civil tenders containing Official Tender Reference Numbers, Work Descriptions, Total Contract Values, Award Dates, and Defect Liability Periods (DLP).
- **Defect Liability Period (DLP) Enforcement:** When a complaint is lodged on a road under an active Defect Liability Period (typically 36 months for bituminous asphalt), the system highlights an active warranty alert. The municipality is legally empowered to order the original contractor to repair the road at zero public cost.
- **Contractor Quality Metrics:** Maintains a record of contractor performance across municipal divisions, highlighting recurring failures during warranty periods.

> **[DEMO / SYNTHETIC PROCUREMENT DATA NOTICE]**
> *DEMONSTRATION NOTICE: Tender records in the project seed database (such as UP-PWD-MRT-2024-JLC-041 and M/s Meerut Highway Infrastructure Pvt. Ltd.) are realistic synthetic models calibrated for the SIH 2026 Internal Hackathon. They do not constitute official UP e-Procurement records.*

23. Geospatial Intelligence & Interactive Mapping System

Geospatial capabilities in ROADGUARD AI ensure rapid, pinpoint defect location:

- **Hardware GPS Geolocation:** Leverages W3C Geolocation API (`navigator.geolocation`) with high accuracy flags, acquiring device coordinates and horizontal accuracy margins.
- **Interactive Leaflet Canvas:** Provides a reactive Leaflet/OpenStreetMap interface where citizens can drag a high-contrast pin to fine-tune damage location when GPS drift occurs inside dense urban corridors.
- **Reverse Geocoding:** Converts raw latitude and longitude coordinates into human-readable street addresses and landmarks using reverse geocoding heuristics.
- **Jurisdictional Snapping:** Coordinates are tested against known municipal road vectors, automatically identifying the responsible authority (UP PWD Provincial Division, Meerut Municipal Corporation, or NHAI PIU Meerut).

24. Progressive Web Application (PWA) & Offline Capability

To guarantee reporting reliability across low-connectivity transit routes, ROADGUARD AI is engineered as a certified Progressive Web Application:

- **Full Installability:** Equipped with a W3C Web App Manifest (`manifest.webmanifest`), responsive theme colors (`#1e3a8a`), and standalone display mode, allowing citizens to install the app on Android, iOS, or desktop without app store friction.
- **Workbox Service Worker Precaching:** Managed via `vite-plugin-pwa` and Workbox, precaching 39 static assets (HTML, JavaScript bundles, CSS stylesheets, and civic icons) to guarantee instant offline app shell boot.
- **IndexedDB Transactional Offline Storage:** Implemented in `src/services/offlineSync.ts`. When a citizen submits a report without network access, the report payload and base64 photograph are stored in IndexedDB (`roadguard_offline_db`) under `pending_reports`.
- **Automatic Background Synchronization:** A reactive network observer listens for connectivity recovery. Upon reconnect, the drainer sequentially uploads queued images, invokes the REST API, and clears synchronized drafts.

25. End-to-End Civic Data Flow Pipeline

The comprehensive data flow from citizen capture to final resolution audit is summarized below:

```
CITIZEN REPORTING STAGE
  1. Citizen opens PWA -> Captures camera photo -> Obtains GPS coordinates -> Selects damage
type.
  2. Client-side quality inspection evaluates clarity -> Dispatches multipart request to
/api/reports.
     [Offline Fallback: Saves to IndexedDB 'roadguard_offline_db' -> Auto-drains on network
reconnect]

SERVER INGESTION & AI TRIAGE STAGE
  3. Multer uploads photo -> StorageService saves to Cloudinary / Supabase / Local disk ->
Returns URL.
  4. Express calls AIService -> Classifies distress type, assesses severity & IRC maintenance
recommendation.
  5. DuplicateDetector calculates Haversine distance -> Detects duplicates (<=80m) & clusters
(<=150m).
  6. RoadRiskEngine computes 6-factor score (0-100) -> Generates explainable diagnostic
rationale.
  7. JurisdictionService snaps incident to nearest RoadSegment & Department (UP PWD, Nagar Nigam,
NHAI).
  8. SLAService computes target resolution hours -> Prisma saves RoadReport, AIAnalysis, Priority
records.

MUNICIPAL ACTION & REPAIR STAGE
```

```
  9. Incident appears on Authority Operations Dashboard, ordered by Dynamic Risk Score.
  10. Executive Engineer reviews report dossier, assigns field maintenance contractor, advances
status.
  11. Maintenance crew executes physical pavement repair (asphalt patch compaction / crack
sealing).

VERIFICATION & LIFECYCLE AUDIT STAGE
  12. Supervising engineer captures and uploads 'After-Repair' photographic proof.
  13. VerificationService executes AI visual comparison -> Computes surface improvement & match
confidence.
  14. Executive Engineer reviews dual-photo comparison -> Signs off 'APPROVED' or
'REJECTED_REINSPECT'.
  15. Status changes to RESOLVED -> StatusTimelineEvent logged -> RoadHealthService updates
corridor score.
```

26. Complete User Journeys (Citizen & Municipal Authority)

A. Detailed Citizen User Journey:

- Step 1 (Discovery): Citizen launches ROADGUARD AI PWA on mobile browser while encountering a hazardous pothole on Jail Chungi Road.
- Step 2 (Quick Authentication): Citizen registers or uses one-click demo login (`citizen@roadguard.demo`).
- Step 3 (Evidence Capture): Taps 'Report Road Hazard' -> Activates native mobile camera -> Captures high-resolution photo of the pavement defect.
- Step 4 (Geocoding & Fine-Tuning): Device auto-detects GPS coordinates (28.9815° N, 77.7380° E). Citizen drags map pin slightly to align with lane position.
- Step 5 (AI Live Preview): Selects damage category 'Pothole'. The optical checker displays 'Image Quality: 92% Excellent | Road Surface Detected'.
- Step 6 (Submission): Clicks 'Submit Road Report'. Server assigns ticket ID `RG-MRT-2026-000101`, assigns Risk Score 82 (CRITICAL), and routes to UP PWD.
- Step 7 (Tracking & Resolution): Citizen monitors `/my-reports`, receives status progression notifications, and views verified after-repair photo upon completion.

B. Detailed Municipal Authority User Journey:

- Step 1 (Operations Console Access): Executive Engineer logs into `/authority` using departmental credentials (`authority@roadguard.demo`).
- Step 2 (Operational Triaging): Reviews KPI cards: 18 Total Complaints, 7 Critical Incidents, 2 Pending AI Verifications. The priority table displays ticket `RG-MRT-2026-000101` at the top with Risk Score 82.
- Step 3 (Dossier Review): Clicks ticket -> Inspects AI diagnosis, IRC remedial advice ('Cold mix or hot bituminous asphalt patch repair with mechanical compaction'), and active tender warranty alert under DLP.
- Step 4 (Contractor Dispatch): Advances status to `IN_PROGRESS` and notifies contractor M/s Meerut Highway Infrastructure.
- Step 5 (After-Repair Verification): Upon repair, uploads contractor post-work photo. AI verification engine generates a Visible Improvement Score of 89/100 and recommends PASS.
- Step 6 (Final Sign-Off): Executive Engineer approves closure -> Ticket marks `RESOLVED` -> Segment health score improves automatically.

27. Comprehensive User Interface & Screen Specifications

ROADGUARD AI features 10 dedicated application pages designed with a sleek civic design system:

Page Name	Route Path	Primary Functional Elements & Widgets	User Interactions & API Calls
Landing Page	/	Civic Hero Banner, Real-Time Statistics Counters, Recent Complaints Carousel, 6-Step Workflow Infographic, CTA Buttons.	Public landing view; loads live summary metrics from `/api/analytics/summary` and recent reports from `/api/reports`.
Login Page	/login	Role-aware login form, password visibility toggle, Demo Quick-Login buttons for Citizen and Authority roles.	Submits credentials to `POST /api/auth/login`; stores signed JWT in localStorage and updates global `AuthContext`.
Register Page	/register	User registration form capturing Full Name, Email, Password, Phone Number, and Role selection.	Submits user details to `POST /api/auth/register`; automatically authenticates upon successful account creation.
Create Report Page	/report	Camera/Gallery File Picker, Image Quality Preview, GPS Geolocation Badge, Draggable Leaflet Pin, Damage Type Grid, Description Field.	Validates image quality client-side; submits multipart data to `POST /api/upload` then `POST /api/reports`; saves to IndexedDB if offline.
My Reports Page	/my-reports	Citizen complaint cards, Status Badges, Dynamic Risk Meters, SLA Countdown Timers, Offline Pending Sync Drawer.	Fetches user records via `GET /api/reports/my-reports`; displays real-time synchronization status and retry controls.
Report Detail Page	/reports/:id	Before/After Side-by-Side Photo Viewer, Image Zoom Modal, AI Diagnostic Breakdown, Dynamic Risk 6-Factor Radar, Timeline, Authority Actions.	Calls `GET /api/reports/:id`; allows authorities to patch status via `PATCH /api/reports/:id/status` and upload after-photos.
Authority Dashboard	/authority	KPI Metrics (Total, Critical, SLA Overdue, Avg Risk), Severity/Status Filters, Priority-Sorted Report Table, Direct Action Modals.	Restricted to AUTHORITY role; calls `GET /api/reports` and `GET /api/analytics/summary`; supports fast inline status progression.
Public Map Page	/map	Full-Screen Interactive Leaflet Map, Color-Coded Risk Markers, Incident Details Drawer, Jurisdiction Boundary Filters.	Fetches geo-tagged reports from `GET /api/reports`; renders interactive clusters and road segment overlays.
Tender Intelligence	/tenders	Public Works Tender Directory, DLP	Calls `GET /api/tenders`; displays

Page		Warranty Badges, Contractor Transparency Cards, Road Segment Links.	contract value, work duration, DLP status, and linked road health scores.
Road Health Page	/road-health	Road Segment Health Cards (0-100), Chronic Hotspot Badges, Health Distribution Chart, Maintenance Action Alerts.	Calls `GET /api/road-health` and `/api/road-health/hotspots`; displays segment metrics and preventive maintenance triggers.

28. Meerut Regional Context & Authentic Demonstration Dataset

To ensure realistic testing during the Smart India Hackathon evaluation, ROADGUARD AI was calibrated with authentic geospatial data from the Meerut municipal zone (Uttar Pradesh). Meerut was selected because it represents a fast-growing National Capital Region (NCR) urban cluster featuring complex multi-agency road jurisdictions (UP PWD, Meerut Nagar Nigam, and NHAI):

- **1. Public Works Department (UP PWD Meerut):** Provincial Division Meerut (Executive Engineer: Er. R.K. Sharma). Responsible for major inter-district arterial corridors.
- **2. Meerut Nagar Nigam (Municipal Corporation):** Municipal Corporation Limits (Chief Engineer: Er. A.K. Verma). Responsible for high-density intra-city residential and commercial streets.
- **3. National Highways Authority of India (NHAI Meerut PIU):** Project Implementation Unit Meerut (Project Director: Shri D.P. Gupta). Responsible for bypass expressways and national highway approaches.

Eight Calibrated Meerut Road Segments:

- 1. Jail Chungi Road (MRT-RD-01 | Length: 3.8 km | Arterial | UP PWD | Health: 52 | Chronic Hotspot)
- 2. Surajkund Road (MRT-RD-02 | Length: 3.2 km | Major District | Nagar Nigam | Health: 61 | Chronic Hotspot)
- 3. Baghpat Bypass Road (MRT-RD-03 | Length: 7.5 km | Arterial | UP PWD | Health: 46 | Critical Deterioration)
- 4. Garh Road (MRT-RD-04 | Length: 5.6 km | Arterial | UP PWD | Health: 78 | Normal Condition)
- 5. Hapur Road (MRT-RD-05 | Length: 4.9 km | Arterial | Nagar Nigam | Health: 69 | Moderate Condition)
- 6. Kankerkheda Main Road (MRT-RD-06 | Length: 4.2 km | Major District | UP PWD | Health: 58 | Chronic Hotspot)
- 7. Civil Lines (MRT-RD-07 | Length: 3.1 km | Local Collector | Nagar Nigam | Health: 84 | Good Condition)
- 8. Malyana Chowki – Sharda Road Link (MRT-RD-08 | Length: 3.7 km | Local | Nagar Nigam | Health: 41 | Critical Deterioration)

Authentic Photographic Assets:

In accordance with strict hackathon ethics, all synthetic demo reports in the seed database use 6 real, reusable CC-licensed road distress photographs sourced from Wikimedia Commons and stored locally in `backend/uploads/demo/`:

- `large-road-pothole.jpg` (Severe deep pothole hazard; CC BY-SA 4.0)
- `driving-through-potholes.jpg` (Vehicle navigation through rutted asphalt; CC BY-SA 3.0)
- `potholes-bengaluru-road.jpg` (Urban road surface crater; CC BY-SA 4.0)
- `potholes-on-road.jpg` (Multiple clustered pavement depressions; CC BY-SA 3.0)
- `waterlogged-pothole.jpg` (Monsoon ponding concealing subgrade pothole; CC BY-SA 4.0)
- `repaired-road-patch.jpg` (Compacted asphalt maintenance patch; CC BY-SA 4.0)

[DATA AUTHENTICITY DISCLAIMER]

DEMONSTRATION DISCLAIMER: These Wikimedia Commons photographs are utilized strictly for prototype visual simulation and do not represent actual citizen-submitted photographs taken inside Meerut city.

29. Deployment Architecture & Infrastructure Topology

ROADGUARD AI is engineered for multi-target deployment across local development, containerized environments, and cloud PaaS architectures:

- **1. Local Development Topology:** Both backend and frontend can be started concurrently via npm scripts (`npm run dev:backend` and `npm run dev:frontend`). The backend boots on `http://localhost:5000` while Vite serves the frontend on `http://localhost:5173`.
- **2. Docker Containerization:** Multi-stage Dockerfiles are present for both tiers (`backend/Dockerfile` and `frontend/Dockerfile`), orchestrated via `docker-compose.yml` with automated container networking, volume mounting for `/uploads`, and database connectivity.
- **3. Render Cloud Blueprint Topology:** The repository includes an infrastructure-as-code Blueprint (`render.yaml`) declaring three linked cloud resources: (1) `roadguard-backend` (Node.js web service running Prisma migrations and seed scripts), (2) `roadguard-frontend` (Static PWA web service hosting Vite rollup bundle), and (3) `roadguard-db` (Managed PostgreSQL 16 database).
- **4. Production HTTPS Topology:** In production, the static frontend communicates over TLS/HTTPS with the backend, which proxies image uploads to Cloudinary or Supabase Storage and connects securely to PostgreSQL over SSL connection strings.

> **[PRODUCTION DEPLOYMENT STATUS DISCLOSURE]**
> *DEPLOYMENT READINESS STATEMENT: Production deployment configuration (render.yaml, Dockerfiles, and build pipelines) has been prepared, validated, and pushed to GitHub (branch: main). However, live production deployment on Render is currently prepared but has not yet been finalized; the application is actively demonstrated in local development mode.*

30. Verification, Testing & Build Validation Matrix

Quality assurance across the codebase has been verified through automated test suites and compiler builds:

Test / Verification Check	Execution Command & Scope	Verified Outcome	Technical Details
Backend Jest Test Suite	npm test --prefix backend	PASS (8 / 8 Tests)	3 test suites executed in 3.06 seconds with 100% pass rate.
Health & API Routes	tests/api.test.ts	PASS (4 Tests)	Verified /api/health, /api/reports, /api/tenders, and /api/road-health endpoints.
AI Provider & Schema	tests/ai.test.ts	PASS (2 Tests)	Validated DemoAIProvider output compliance with strict Zod schemas.
Road Risk Priority Engine	tests/priority.test.ts	PASS (2 Tests)	Verified critical escalation for arterial hazards and low scoring for minor ruts.

Frontend Production Build	npm run build --prefix frontend	PASS (0 Errors)	1,647 modules transformed in 4.33s; bundle gzipped to 130.4 kB.
PWA Service Worker Build	vite-plugin-pwa rollup	PASS (39 Precached)	Generated dist/sw.js and workbox-5265bac8.js precaching 544.59 KiB.
Backend TypeScript Check	npm run build --prefix backend	PASS (0 Errors)	Prisma client generated and TypeScript source compiled cleanly to dist/.
Relational DB Seeding	npm run seed --prefix backend	PASS (Seeded)	Successfully populated 3 departments, 8 road segments, 5 tenders, and 32 reports.

31. System Performance, Scalability & Resilience

ROADGUARD AI is architected to handle municipal workloads ranging from small nagar panchayats to megacity municipal corporations:

- **Stateless Backend Architecture:** The Node.js Express server maintains no in-memory session state, enabling horizontal scaling across multiple container instances behind load balancers.
- **Relational Database Optimization:** The Prisma schema specifies compound and single-column B-tree indexes on high-frequency query fields (`userId`, `roadSegmentId`, `departmentId`, `status`, `severity`, `riskScore`, and composite `[latitude, longitude]`), ensuring sub-10ms spatial lookup latencies.
- **Decoupled Object Storage:** Offloading heavy photographic evidence to cloud object storage (Cloudinary / Supabase) protects backend servers from disk exhaustion and leverages global CDN edges for rapid image delivery.
- **Edge Asset Caching:** Service Worker precaching eliminates redundant asset requests, reducing repeat visitor network payload to near zero.

32. Multidimensional Feasibility Analysis

The feasibility of ROADGUARD AI was evaluated across four critical dimensions:

- **1. Technical Feasibility:** High. Built entirely using standard, open-source technologies (React, Node.js, Express, TypeScript, Prisma, PostgreSQL). Zero exotic hardware or proprietary operating systems required. Operates seamlessly on low-cost smartphones.
- **2. Operational Feasibility:** High. Designed to empower municipal Junior Engineers and contractors rather than disrupting established administrative chains. Incorporates familiar IRC maintenance terminologies and respects official departmental hierarchies.
- **3. Economic Feasibility:** Exceptional. Open-source core incurs zero licensing fees. The platform delivers massive cost savings by enforcing contractor Defect Liability Periods (DLP), preventing municipal exchequers from paying for premature road failures.
- **4. Scalability Feasibility:** High. Modular multi-tenant architecture allows new municipal corporations, smart cities, and state PWD divisions to be onboarded simply by adding new Department and RoadSegment records.

33. Socio-Economic Impact & Civic Value Proposition

ROADGUARD AI delivers tangible societal and economic returns across all civic stakeholders:

- **Impact on Citizens & Commuters:** Drastically reduces two-wheeler fatalities, prevents vehicular suspension and tire blowouts, provides transparent tracking, and closes the civic grievance communication loop.
- **Impact on Municipal Authorities:** Replaces chaotic complaint piles with an objective, risk-prioritized work queue, enabling engineers to allocate maintenance budgets based on measurable safety impacts rather than subjective pressure.
- **Impact on Public Procurement:** Transforms public contracting culture by ending contractor impunity. Enforcing 36-month defect liability warranties ensures contractors execute durable road pavement from day one.
- **Impact on Urban Road Infrastructure:** Identifies recurring structural failure corridors, enabling civic administrations to transition from reactive pothole patching to planned, cost-effective full-depth reconstruction.

34. Core Innovation & Key Differentiators (USP)

Rather than creating another generic civic complaint app, ROADGUARD AI introduces a proprietary combination of innovations:

- **1. Dynamic 6-Factor Risk Prioritization Engine:** Unlike basic apps that rely on subjective citizen ratings, ROADGUARD AI calculates an explainable 6-factor risk index weighing pavement severity, direct hazard, traffic corridor class, density, recurrence, and SLA urgency.
- **2. Spatial Deduplication & Radius Clustering:** Automated Haversine spatial proximity algorithms merge co-located complaints into unified tickets, eliminating duplicate work orders.
- **3. Public Tender Intelligence & DLP Enforcement:** The first civic platform to directly cross-reference road damage coordinates against public procurement tenders and contractor Defect Liability Periods (DLP).
- **4. Evidence-Based Dual-Photo Verification:** Enforces strict before/after dual-photo evidence, employing computer vision match scoring to eliminate fake 'paper resolutions'.
- **5. Resilient Offline-First PWA Subsystem:** Guarantees zero citizen grievance loss across low-connectivity rural or urban transit corridors via client-side IndexedDB caching and background sync.

35. Technical Limitations & Operational Constraints

To maintain technical honesty, the project acknowledges four current operational constraints:

- **1. Heuristic Fallback in Demo Mode:** In the current demonstration environment, the system utilizes DemoAIProvider rule-based heuristics rather than a dedicated on-device convolutional neural network (CNN).
- **2. Camera Lighting & Lens Sensitivity:** Optical quality assessment and visual damage classification can be impacted by night-time photography, poor smartphone lenses, or extreme rain glare.
- **3. Urban Canyon GPS Drift:** In dense urban high-rise canyons, standard smartphone GPS can exhibit a 10 to 30 meter horizontal drift, requiring citizen manual pin adjustment.
- **4. Mandatory Human Oversight:** While AI verification calculates surface improvement scores, final closure mandates human-in-the-loop sign-off to ensure municipal compliance.

36. Future Roadmap & Advanced Research Enhancements

The YuvaTech engineering roadmap includes several planned enhancements for subsequent platform iterations:

- **1. On-Device Edge Computer Vision (YOLOv8):** Deploying a lightweight, quantized YOLOv8 object detection model directly into the browser via WebAssembly (Wasm) and TensorFlow.js for instantaneous bounding-box pavement distress detection.
- **2. Automated Vehicular Telematics & Inertial Sensing:** Leveraging mobile smartphone accelerometers and gyroscopes mounted on commuter vehicles or public transit buses to automatically detect and map road roughness and pothole jolts without manual photo capture.
- **3. National Civic ERP Integration:** Direct bidirectional API connectors linking ROADGUARD AI with national and state civic portals (CPGRAMS, Jansunwai UP, e-NagarSewa) and government e-marketplace (GeM) portals.
- **4. Civic Gamification & Community Recognition:** Introducing citizen contribution karma, verified civic badges, and community leaderboards to incentivize proactive neighborhood road hazard reporting.
- **5. Satellite & Aerial Orthophoto Analysis:** Integrating high-resolution municipal satellite and dashcam street-level imagery to track macro-level road deterioration across entire municipal networks.

37. Concluding Engineering Evaluation

ROADGUARD AI successfully demonstrates how modern web engineering, geospatial intelligence, explainable algorithmic prioritization, and civic procurement transparency can converge to solve one of India's most urgent infrastructure challenges.

Developed for the Smart India Hackathon (SIH) Internal Hackathon under Problem Statement MB-04 ('Smart Infrastructure & Logistics') by Team YuvaTech, the platform transitions road maintenance from a fragmented, reactive complaint process into an accountable, evidence-driven, and preventive lifecycle system. By combining mobile PWA offline reporting, dynamic multi-factor risk scoring, spatial deduplication, public tender defect liability tracking, and dual-photo AI verification, ROADGUARD AI provides a comprehensive, production-ready blueprint for safer Indian roads.

38. Academic Literature & Technical References

The engineering design and technical implementation of ROADGUARD AI are grounded in the following authentic standards, documentation, and research literature:

- 1. Indian Roads Congress (IRC), 'Guidelines for the Design of Flexible Pavements for Low Volume Rural Roads', IRC:SP:72-2015, New Delhi, India.
- 2. Indian Roads Congress (IRC), 'Code of Practice for Road Signs', IRC:67-2012, New Delhi, India.
- 3. Ministry of Road Transport and Highways (MoRTH), 'Road Accidents in India 2022', Transport Research Wing, Government of India, New Delhi.
- 4. Arya, D. et al., 'Global Road Damage Detection: State-of-the-Art Solutions and Open Challenges', IEEE Transactions on Intelligent Transportation Systems, RDD2022 Benchmark.
- 5. React Core Team, 'React 18 Documentation & Concurrent Features', https://react.dev
- 6. Vite Community, 'Vite Next Generation Frontend Tooling Documentation', https://vite.dev
- 7. Express.js Project, 'Express Web Framework for Node.js API Reference', https://expressjs.com
- 8. Prisma Data Inc., 'Prisma ORM Technical Reference & Client Documentation', https://www.prisma.io/docs
- 9. PostgreSQL Global Development Group, 'PostgreSQL 16 Documentation', https://www.postgresql.org/docs
- 10. Leaflet Project, 'Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps', https://leafletjs.com

- 11. Google Chrome Developers, 'Workbox: Production-Ready Service Worker Libraries', https://developer.chrome.com/docs/workbox
- 12. Render Cloud Inc., 'Render Blueprint Specification & Infrastructure as Code Guide', https://render.com/docs/blueprint-spec

39. Technical Appendix (Repository Tree, Config & Environment)

Appendix A: Complete Workspace File Structure

```
roadguard-ai/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma            # 10 Prisma Relational Models
│   │   └── seed.ts                  # Meerut Calibration Seed Data
│   ├── scripts/
│   │   └── prepare-prisma.js        # Cross-Platform Schema Switcher
│   ├── src/
│   │   ├── config/prisma.ts         # Prisma Client Singleton
│   │   ├── controllers/             # 6 REST API Controllers
│   │   ├── middleware/              # Auth & Error Interceptors
│   │   ├── routes/                  # 9 Domain Route Handlers
│   │   ├── services/                # 10 Domain Business Services
│   │   └── server.ts                # Express Server Entrypoint
│   ├── tests/                       # 3 Jest Test Suites (8 Tests)
│   ├── uploads/demo/                # 6 Wikimedia Commons Photographs
│   ├── Dockerfile                   # Multi-Stage Node.js Dockerfile
│   └── package.json                 # Backend Dependencies
├── frontend/
│   ├── public/                      # PWA Manifest & Civic Icons
│   ├── src/
│   │   ├── components/              # 12 Component Subdomains
│   │   ├── context/                 # Global Auth & Role Session
│   │   ├── pages/                   # 10 Application Pages
│   │   ├── services/                # REST API & IndexedDB Offline Sync
│   │   ├── App.tsx                  # React Router Configuration
│   │   └── main.tsx                 # React DOM Bootstrapper
│   ├── Dockerfile                   # Static Frontend Dockerfile
│   ├── vite.config.ts               # Vite & Workbox PWA Plugin
│   └── package.json                 # Frontend Dependencies
├── documentation/                   # Submission-Ready Documentation
├── scripts/                         # Automated Documentation Generators
├── docker-compose.yml               # Multi-Container Topology
├── render.yaml                      # Render Cloud Infrastructure Blueprint
└── package.json                     # Workspace Root Scripts
```

Appendix B: Environment Variable Configuration Reference (Names Only)

- `NODE_ENV` — Environment flag ('development' or 'production').
- `PORT` — Express API server port (default: 5000).
- `DATABASE_URL` — Connection URI for SQLite (local) or PostgreSQL (production).
- `JWT_SECRET` — Cryptographic HMAC-SHA256 secret for token signing (never exposed).
- `JWT_EXPIRES_IN` — Token lifespan duration (default: '7d').
- `AI_PROVIDER` — AI strategy selector ('demo', 'gemini', or 'openai').
- `GEMINI_API_KEY` — Google Gemini Vision API key (optional; cloud mode).

- `OPENAI_API_KEY` — OpenAI GPT-4o Vision API key (optional; cloud mode).
- `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET` — Cloudinary credentials (optional).
- `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` — Supabase Storage credentials (optional).
- `VITE_API_URL` — Client-side API root endpoint for frontend requests.

Appendix C: Test Execution Verification Log Summary

```
Test Suites: 3 passed, 3 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        3.059 s
Files:
  - tests/api.test.ts (PASS - 4 tests)
  - tests/ai.test.ts (PASS - 2 tests)
  - tests/priority.test.ts (PASS - 2 tests)
Frontend Build: 1,647 modules transformed in 4.33s; 39 PWA assets precached.
```